Code Explanation

Code Explanation: AWS Glue PySpark ETL JobThis AWS Glue job implements a complete ETL pipeline for customer order processing with Slowly Changing Dimension (SCD) Type 2 logic, data cleaning, aggregations, and catalog registration.---

Step 1: Configuration Loading

Function: `get_job_parameters()`- Returns a comprehensive configuration dictionary containing: - **Input paths**: S3 locations for customer data, order data, and previous customer state - **Output paths**: Destinations for curated data, state snapshots, analytics, and catalog tables - **Catalog settings**: Glue database and table names - **Hudi configuration**: Settings for Apache Hudi table format (used for SCD Type 2) - **Processing rules**: Null handling, duplicate removal, CSV parsing options - **SCD Type 2 settings**: Column names and comparison fields for change detection---

Step 2: Spark Context Initialization

Function: `initialize_spark_contexts()`- Detects the runtime environment (test vs. production)- In production: Retrieves job name from Glue job parameters- Creates SparkContext, GlueContext, and Job objects- Returns initialized contexts for use throughout the pipeline---

Step 3: Data Reading (DataReader Class)

Purpose: Safe data ingestion with error handling**Key Methods:**- `_read_csv()`: Reads CSV files with header and delimiter options- `_read_parquet()`: Reads Parquet files- `read_data_safe()`: Wrapper that catches exceptions and returns None if path doesn't exist**Usage:** Prevents job failure when optional input paths (like previous state) don't exist yet---

Step 4: Data Writing (DataWriter Class)

Purpose: Safe data output with error handling**Key Methods:**- `_write_parquet()`: Writes Parquet format- `_write_csv()`: Writes CSV format with headers- `write_data_safe()`: Wrapper that catches exceptions and logs success/failure---

Step 5: Schema Definition

Functions: `get_customer_schema()` and `get_order_schema()`- Define explicit schemas for input data validation- **Customer schema**: CustId (required), Name, EmailId, Region- **Order schema**: OrderId (required), CustId (required), OrderDate, Amount---

Step 6: Data Cleaning

Function: `remove_nulls_and_duplicates()`**Process:**1. Iterates through all columns2. Filters out rows where any column contains the string literal "Null" OR is actually NULL3. Removes duplicate rows (full row comparison)4. Returns cleaned DataFrame**Implements:** TR-CLEAN-001 and TR-CLEAN-002 requirements---

Step 7: Catalog Registration

Function: `write_to_catalog()`**Process:**1. Writes DataFrame to S3 in specified format (Parquet/CSV)2. Constructs full S3 path using base path + table name3. Logs catalog registration

(actual registration happens via Glue crawler or API)**Implements:** TR-CATALOG-001 and TR-CATALOG-002 requirements---

Step 8: Change Detection

Function: `detect_customer_changes()`**Process:**1. Handles initial load case (no previous data → all records are new)2. Normalizes column names to lowercase3. Performs left anti-join to find new customers4. Compares specified columns (Name, EmailId, Region) to detect changes5. Returns only customers with changes**Logic:** Uses OR condition across comparison columns to detect any attribute change---

Step 9: SCD Type 2 Implementation

Function: `apply_scd_type2()`**Process:**1. Adds SCD Type 2 metadata columns to current data: - `IsActive`: Boolean flag (True for current records) - `StartDate`: Timestamp when record became active - `EndDate`: NULL for active records, timestamp when expired - `OpTs`: Operation timestamp for versioning2. **Initial Load Path:** If no previous data exists, returns all records as active3. **Incremental Load Path:** - Detects changed customers using `detect_customer_changes()` - Expires old versions by setting `IsActive=False` and `EndDate=current_timestamp` - Keeps unchanged active records as-is - Keeps all historical (already inactive) records - Adds new versions of changed customers with `IsActive=True`4. Unions three DataFrames: unchanged + expired + new versions**Implements:** TR-SCD-001 requirement---

Step 10: Aggregation

Function: `calculate_customer_aggregate_spend()`**Process:**1. Normalizes column names2. Casts Amount column to double for numeric operations3. Performs inner join between customer and order data on CustId4. Groups by CustId and OrderDate5. Calculates sum of Amount as TotalSpend**Output:** Customer spending totals per date**Implements:** TR-AGG-001 requirement---

Step 11: Hudi Table Writing

Function: `write_hudi_table()`**Process:**1. Configures Apache Hudi options: - Record key: CustId (unique identifier) - Precombine field: OpTs (for deduplication) - Table type: COPY_ON_WRITE (optimized for read-heavy workloads) - Operation: UPSERT (insert or update) - Hive sync: Enabled for Glue catalog integration2. Writes DataFrame in Hudi format3. **Fallback:** If Hudi write fails, falls back to regular Parquet format**Purpose:** Enables efficient SCD Type 2 queries and time-travel capabilities---

Step 12: Main Pipeline Execution

Function: `main()`**Complete Pipeline Flow:**1. **Initialize:** Set up Spark contexts and load configuration2. **Ingest:** - Read customer CSV from S3 (TR-INGEST-001) - Read order CSV from S3 (TR-INGEST-002) - Normalize column names to lowercase3. **Clean:** - Remove nulls and duplicates from customer data (TR-CLEAN-001) - Remove nulls and duplicates from order data (TR-CLEAN-002)4. **Catalog:** - Write cleaned customer data to Glue catalog (TR-CATALOG-001) - Write cleaned order data to Glue catalog (TR-CATALOG-002)5. **SCD Type 2:** - Read previous customer state (if exists) - Apply SCD Type 2 transformations (TR-SCD-001) - Write ordersummary table in Hudi format - Save current customer state for next run6. **Aggregate:** - Calculate customer aggregate spend (TR-AGG-001) - Write aggregated data to analytics location and catalog7. **Finalize:** Commit Glue job---

Key Design Patterns

1. **Safe Operations:** All I/O operations wrapped in error handling2. **Idempotency:** Job can be re-run safely; SCD Type 2 handles duplicates3. **Incremental Processing:** Only processes changed customer records4. **State Management:** Saves customer state between runs for change detection5. **Fallback Mechanisms:** Hudi write falls back to Parquet if needed6. **Separation of Concerns:** Reader, Writer, and transformation logic separated into distinct functions/classes---

Data Flow Summary

```
Raw CSV Data (S3) ↓Clean & Normalize ↓Write to Catalog (Curated Layer) ↓Apply SCD Type 2 Logic ↓Write Hudi Table (ordersummary) ↓Calculate Aggregations ↓Write Analytics Data (customeraggregatespend)
```